

Mechatronics & Robotics Engineering Department

Microcontroller Engineering I

Light Dimmer System Using TRIAC Control

Prepared By

Student Name	ID
Batool Khaled	9500
Habiba Ali	9628
Yahia Haitham	9613
Youssif Ahmed	9522

Instructor:

Dr. Hossam Eldin Mostafa

Table of Contents

I.	Introduction	3
II.	Hardware	3
III.	How the system works.....	4
IV.	Code	5
4.1	ATmega32A Code	5
4.2	ESP32 Code	15
V.	Circuit Schematic.....	16
VI.	Challenges Faced.....	16
VII.	Conclusion.....	17

I. Introduction

The light dimmer system is an electronic circuit used to control the brightness of an AC lamp by regulating the power delivered to the load. The dimmer change the RMS voltage supplied to the lamp to give it a smooth brightness control.

In this project the brightness is controlled using three different methods:

- 1) Potentiometer control
- 2) Keypad control
- 3) Serial control using ESP32

Main project objects are:

- 1) Design the light dimmer using TRIAC
- 2) Control system using keypad and potentiometer
- 3) Use interrupts for accurate timings and control
- 4) An LCD that shows the output of the voltage at a given moment
- 5) Use a zero crossing detection circuit

II. Hardware

The hardware components used were as follows

- 1) Microcontroller
ATmega 32A
- 2) AC/DC Step Down Power Supply Module
This module converts the high AC voltage into low regulated DC voltage suitable for the hardware components.
 - ATmega32A
 - LCD
 - Optocouplers
- 3) TRIAC (BT136)
TRIAC allows the current to pass during selected durations of the AC waveform. It changes the trigger delay after each zero-crossing stage, changing the RMS voltage value.
The smaller the firing angle, the higher the power, the brighter the lamp.
- 4) Optocoupler (MOC3021)
Detect the zero in sine wave to trigger the TRIAC.

5) Potentiometer

Provides an analog control for the lamp where it's connected to ADC input as the knob is rotated the ADC determines the firing angle.

Connected to ADC pins.

6) Keypad

The keypad is connected and programmed in the code so that it works every 5 digits; for example, when the user enters 21 it approximates it to 20 when 24 it is approximated to 25.

7) ESP32

Used to control the lamp brightness by USART serial communication.

8) Snubber Circuit

Consists of $39\ \Omega$ resistor and a $0.01\ \mu\text{F}$ capacitor.

It reduces voltage spikes and protects the TRIAC.

9) LCD

It displays the brightness percentage, the system status and the operating mode.

III. How the system works

The system uses the AC/DC step down module to convert the AC voltage to regulated DC voltage for hardware components.

Then the AC is connected to a zero cross detection circuit using the optocoupler, whenever the waveform crosses zero volt it generates a pulse and sends it to the interrupt pin in our microcontroller.

Next step is where the user controls the brightness either using the potentiometer, keypad or ESP32 according to the mode selected, where the microcontroller reads these inputs and give the corresponding voltage to it.

After each zero-cross interrupt, the ATmega32A calculates a delay time according to the selected brightness level.

IV. Code

4.1 ATmega32A Code

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "my_lcd.h"

#define BAUD_PRESCALE (((F_CPU / (USART_BAUDRATE * 16UL))) - 1)

/* ----- Variables ----- */

volatile uint8_t zero_detected = 0;
volatile uint8_t flag2 = 0;
volatile uint8_t flagint2 = 0;
volatile uint8_t mode2 = 0;
volatile uint8_t mode3 = 0;
volatile uint8_t mode = 1;
volatile uint8_t flag3 = 0;
volatile uint8_t flag_in_mode3 = 0;
volatile uint8_t mode1 = 1;
volatile uint8_t flag_in_mode2 = 0;
volatile uint8_t found_index = 0;
volatile uint8_t i = 0;
volatile uint8_t flag_mode2 = 0;
volatile uint16_t delay_ticks = 0;
char message[17] = "";
char message1[17] = "";
char show[17] = "";
uint8_t v = 0;
uint8_t vmode2 = 0;
uint8_t vold = 255;
uint8_t VESP = 0;
```

```
uint8_t vmode1 = 255;
uint8_t pos = 0;
uint8_t p = 0;
uint8_t p2 = 0;
uint8_t flag_relay = 0;
uint32_t valueesp = 0;
char letter = 0;

const char keys[4][4] = {
    {'1', '2', '3', 'A'},
    {'4', '5', '6', 'B'},
    {'7', '8', '9', 'C'},
    {'*', '0', '#', 'D'}
};

const char letters[21] = {
    'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i',
    'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u'
};

/* ----- Delay Table ----- */
const unsigned int arr[21] = {
    9800, // 0%   -> value 0
    9350, // 5%   -> value 11
    8900, // 10%  -> value 22
    8450, // 15%  -> value 33
    8000, // 20%  -> value 44
    7550, // 25%  -> value 55
    7100, // 30%  -> value 66
    6650, // 35%  -> value 77
    6200, // 40%  -> value 88
    5750, // 45%  -> value 99
    5300, // 50%  -> value 110
    4850, // 55%  -> value 121
    4400, // 60%  -> value 132
    3950, // 65%  -> value 143
    3500, // 70%  -> value 154
}
```

```
    3050, // 75%  -> value 165
    2800, // 80%  -> value 176
    2300, // 85%  -> value 187
    1900, // 90%  -> value 198
    1250, // 95%  -> value 209
    800 // 100% -> value 220
};

/* ----- Keypad Read ----- */
char keypad_getkey(void) {
    for (uint8_t row = 0; row < 4; row++) {
        PORTA |= 0xF0; // all rows HIGH
        PORTA &= ~(1 << (row + 4)); // one row LOW

        for (uint8_t col = 0; col < 4; col++) {
            if (!(PINB & (1 << (col + 4)))) {
                _delay_ms(20);
                while (!(PINB & (1 << (col + 4))));
                return keys[row][col];
            }
            _delay_ms(20);
        }
    }
    return 0;
}

uint8_t UART_dataAvailable(void) {
    return (UCSRA & (1 << RXC));
}

// Non-blocking read - returns -1 if no data available
int UART_getChar_NonBlocking(void) {
    if (UCSRA & (1 << RXC))
        return UDR;
    else
        return -1;
}
```

```
void UART_init(long USART_BAUDRATE) {
    UBRRH = (BAUD_PRESCALE >> 8); // Load upper 8-bits of the baud rate
    UBRRL = BAUD_PRESCALE; // Load lower 8-bits
    UCSRB = (1 << RXEN) | (1 << TXEN); // Turn on transmission and reception
    UCSRC = (1 << URSEL) | (1 << UCSZ0) | (1 << UCSZ1); // Use 8-bit char size
}

int UART_getChar(FILE *stream) {
    while ((UCSRA & (1 << RXC)) == 0); // Wait till data is received
    return (UDR); // Return the byte, reading UDR clears RXC flag automatically
}

int UART_putChar(char c, FILE *stream) {
    while ((UCSRA & (1 << UDRE)) == 0); // Wait until buffer is empty
    UDR = c; // Writing to UDR clears UDRE flag automatically
    return 0;
}

static FILE uart_str = FDEV_SETUP_STREAM(UART_putChar, UART_getChar, _FDEV_SETUP_RW);

/* ----- Interrupts ----- */
ISR(INT1_vect) {
    zero_detected = 1;
}

ISR(INT0_vect) {
    flag_relay = 1;
}

ISR(INT2_vect) {
    GICR &= ~(1 << INT1);
    _delay_ms(30);
    if (!(PINB & (1 << PB2))) {
        // Reset flag_in_mode2 but don't let it affect mode cycling
        if (flag_in_mode2 == 1) {
            flag_in_mode2 = 0;
            mode = 1;
        }
    }
}
```

```
mode = mode + 1;

if (mode > 3) {
    mode = 1;
}

// Reset all mode flags

mode1 = 0;
mode2 = 0;
mode3 = 0;
flag_mode2 = 0;
zero_detected = 0;
LCD_Clear();

if (mode == 1) {
    mode1 = 1;
    mode2 = 0;
    mode3 = 0;
    flag2 = 0;
    flag3 = 1;
    LCD_Gotoxy(0, 0);
    LCD_String("Dimmer Mode1 ADC");
    p = 5 * v;
    sprintf(message, "Out=%3u Percent", p);
    LCD_String_xy(1, 0, message);

} else if (mode == 2) {
    mode1 = 0;
    mode2 = 1;
    mode3 = 0;
    LCD_Gotoxy(0, 0);
    LCD_String("Dimmer Mode2 Pad");
    pos = 0;
    show[0] = '\0';

} else if (mode == 3) {
    mode1 = 0;
    mode2 = 0;
```

```
        mode3 = 1;

        flag2 = 0;

        flag3 = 1;

        LCD_Gotoxy(0, 0);

        LCD_String("Dimmer Mode3 ESP");

        p2 = VESP * 5;

        sprintf(message1, "Out=%3u %%", p2);

        LCD_String_xy(1, 0, message1);

    }

}

GIFR |= (1 << INTF2);
GICR |= (1 << INT1);
}

/* triac pulse */
ISR(TIMER1_COMPA_vect) {
    PORTA |= (1 << PA0);

    for (volatile uint8_t i = 0; i < 50; i++);

    PORTA &= ~(1 << PA0);

    TCCR1B = 0;

    if (vold != v) {
        p = 5 * v;

        sprintf(message, "%3u", p);

        LCD_String_xy(1, 4, message);

        2 vold = v;
    }
}

/* ----- ADC ----- */
uint8_t ReadVoltage(void) {
    ADCSRA |= (1 << ADSC);

    while (ADCSRA & (1 << ADSC));

    ADCSRA |= (1 << ADIF);

    return ((uint32_t) ADCW * 20UL) / 1023;
}
```

```
}

/* ----- MAIN ----- */

int main(void) {
    /* INT1 input */
    DDRD &= ~(1 << PD3);
    PORTD |= (1 << PD3);

    UART_init(9600); // or whatever baud rate your ESP uses
    stdout = stdin = &uart_str;
    DDRD |= (1 << PD6);
    PORTD |= (1 << PD6);
    unsigned char cmd = 0;
    /* INT2 input */
    DDRB &= ~(1 << PB2);
    PORTB |= (1 << PB2);

    /* interrupts */
    /* LCD */
    LCD_Init();
    LCD_Clear();
    LCD_Gotoxy(0, 0);
    LCD_String("Dimmer Mode1 ADC");

    sprintf(message, "Out=%3u Percent", 2 * v);
    LCD_String_xy(1, 0, message);

    /* Keypad rows = PA4..PA7 output */
    DDRA |= 0xF0;

    /* Triac gate = PA0 output */
    DDRA |= (1 << PA0);
    PORTA &= ~(1 << PA0);

    /* Columns = PB4..PB7 input pullup */
    DDRB &= ~0xF0;
    PORTB |= 0xF0;
```

```
/* ADC CH3 */

ADMUX = 0x03;

ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (0 << ADPS0) | (0 << ADSC) | (0 << ADIF) | (0 << ADIE);

SFIOR = (0 << ADTS2) | (0 << ADTS1) | (0 << ADTS0);

/* Timer1 CTC */

TCCR1A = 0;
TCCR1B = 0;
TCCR1B |= (1 << WGM12);
TIMSK |= (1 << OCIE1A);
GICR |= (1 << INT1) | (1 << INT2) | (1 << INT0);
MCUCSR &= ~(1 << ISC2); // falling edge INT2
MCUCR |= (1 << ISC11) | (1 << ISC10); // rising edge INT1
MCUCR |= (1 << ISC01) | (0 << ISC00);
sei();

while (1) {
    if (mode1) {
        GICR &= ~(1 << INT1);
        vmode1 = ReadVoltage();
        GICR |= (1 << INT1);
    } else if (mode2) {
        flag_mode2 = 0;
        pos = 0;
        flag2 = 1;
        show[0] = '\0';
        LCD_Clear();
        LCD_Gotoxy(0, 0);
        LCD_String("Dimmer Mode2 Pad");

        while (flag2) {
            letter = keypad_getkey();
            if (letter != 0) {
                if (letter == '#') {
```

```
        flag2 = 0;
        flag_in_mode2 = 1;
        vmode2 = atoi(show);

        if (vmode2 > 100) {
            vmode2 = 100;
        }
        vmode2 = ((vmode2 + 2) / 5) * 5;
        vmode2 = vmode2 / 5;
        flag_mode2 = 1;
        mode2 = 0;
        p2 = vmode2 * 5;
    } else if ((letter == '*')) {
        flag_mode2 = 0;
        pos = 0;
        flag2 = 1;
        show[0] = '\0';

        LCD_Clear();
        LCD_Gotoxy(0, 0);
        LCD_String("Dimmer Mode2 Pad");
        continue;
    } else {
        show[pos++] = letter;
        show[pos] = '\0';
    }
    sprintf(message1, "Out=%s %%", show);
    LCD_String_xy(1, 0, message1);
}

GICR |= (1 << INT1);
sprintf(message1, "Out=%3u %%", p2);
LCD_String_xy(1, 0, message1);

} else if (mode3) {
    GICR &= ~(1 << INT1);
```

```
cmd = UART_getChar_NonBlocking();
if (cmd != -1) {
    for (uint8_t i = 0; i < 21; i++) {
        if (letters[i] == cmd) {
            if (VESP != i) {
                VESP = i;
            }
            break;
        }
    }
}
GICR |= (1 << INT1);
}
if (zero_detected) {
    zero_detected = 0;
    TCNT1 = 0;
    TCCR1B |= (1 << CS11);
    if (flag_relay == 1) {
        _delay_ms(50);
        if (!(PIND & (1 << PIND2))) {
            PORTD ^= (1 << PD6);
        }
        while (!(PIND & (1 << PIND2))) {
            _delay_ms(10);
        }
        _delay_ms(50);
        flag_relay = 0;
    }
    vold = v;
    if (mode1) {
        v = vmode1;
    } else if (flag_mode2) {
        v = vmode2;
    } else if (mode3) {
        v = VESP;
    }
}
```

```
        if (v > 20) v = 20;

        delay_ticks = arr[v];

        OCR1A = delay_ticks * 2;

    }

}

}
```

4.2 ESP32 Code

```
#define BLYNK_TEMPLATE_ID "TMPL2toWrkVxJ"
#define BLYNK_TEMPLATE_NAME "Dimmer2"
#define BLYNK_AUTH_TOKEN "VnK5LRgbZR8vOrSJJe3mdMt8uhWKnLjoK"
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

char ssid[] = "Ahmed";
char pass[] = "01551810300";
char scaled = 0;
char sent = 0;
int i =0;
const char letters[21] = {
    'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i',
    'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u'
};

void setup() {
    Serial.begin(9600);
    Serial2.begin(9600, SERIAL_8N1, 16, 17);
    Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
}

BLYNK_WRITE(V0) {
    int value = param.asInt();
    scaled = (value * 20) / 1023;
    Serial.println(letters[scaled]);
    Serial2.write(letters[scaled]);
}
```

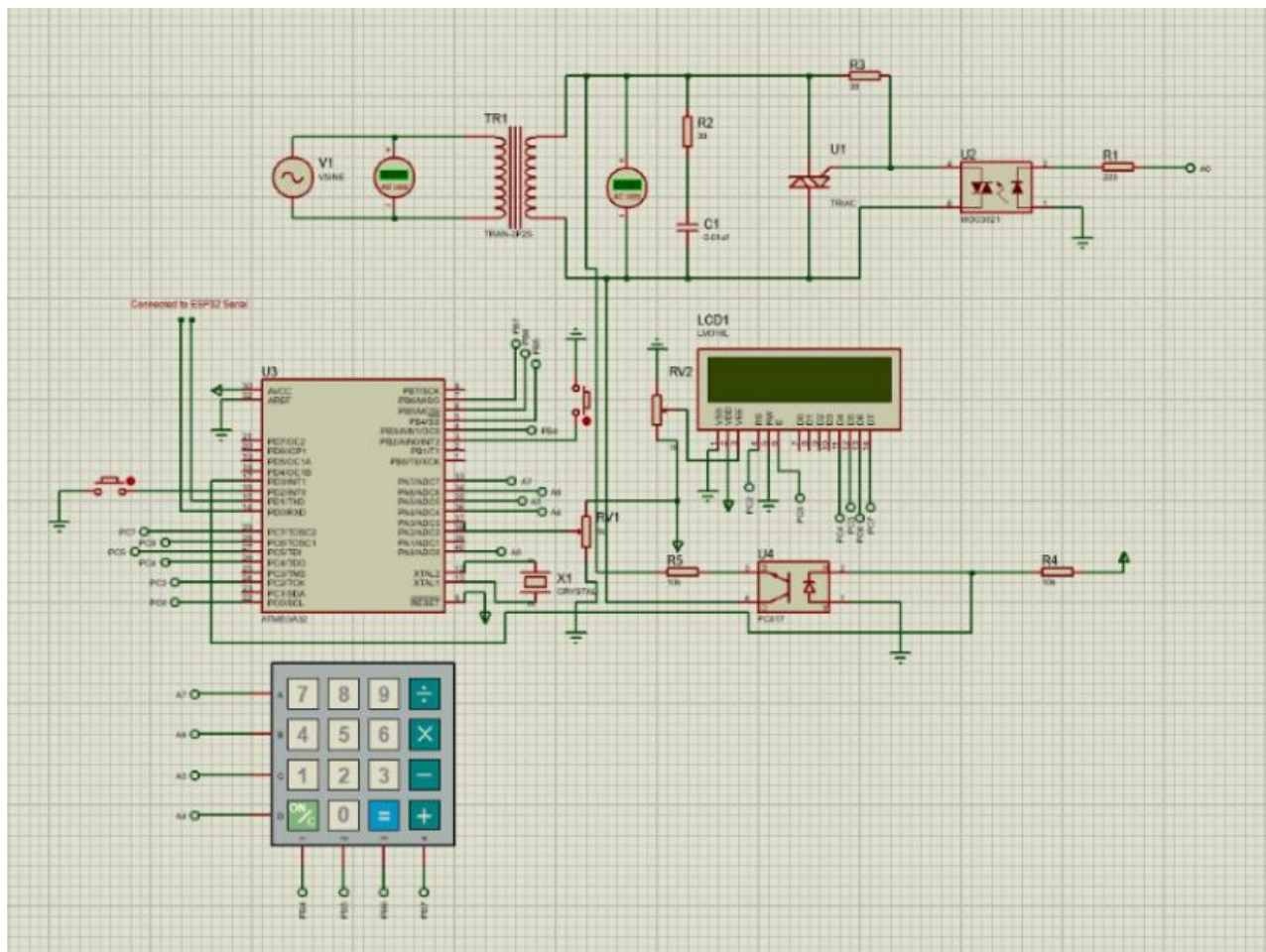
```

    sent = letters[scaled];
}

void loop() {
    Blynk.run();
}

```

V. Circuit Schematic



VI. Challenges Faced

Limiting every function in the system to work under 10ms.

Power used was an issue as a relay module was used, 2 lamps and an LCD. Therefore, the system was optimized to minimize voltage drop and ensure stable electrical performance.

VII. Conclusion

The system successfully combined

- Analog control using a potentiometer
- Digital control using a keypad
- Interrupt-based synchronization
- USART communication

Achieving smooth and reliable brightness control system that can be later expanded to a complete light system.